

5-Mark Questions & Answers - Data Structures Module 1

Topic 1: Concept of ADT

Q1. Explain ADT in detail. How is an ADT implemented in a programming language? Give an example of Stack ADT with its operations.

1. Detailed Concept of ADT

An Abs

2. Implementation in Programming Languages

In C/C+

3. Stack ADT Example & Operations

A Stack

// Interface Definition (Stack ADT)

typedef

// Core Operations:

void pu

int pop(Stack *s) {

if (s->to

int peek(Stack *s) {

return (

Q2. Describe how ADT provides abstraction and information hiding with a real-life example (e.g., List ADT or Queue ADT).

1. Abstraction and Information Hiding

* **A
operatio

2. Real-Life Example: Queue ADT (Ticket Counter Analogy)

Think o

Front

Rear

+-----+

Topic 2: Types of Data Structures

Q3. Explain the classification of data structures in detail with examples and diagrams.

Data structures are categorized into two primary hierarchies:

Data Structures

|

1. Primitive Data Structures

Base d

2. Non-Primitive Data Structures

Comple

Q4. Discuss the various types of data structures along with their real-world applications.

| Data Structure | Type | Key Characteristics | Real-World Application |

| :--- | :-

Topic 3: Operations on Data Structures

Q5. Explain in detail the various operations performed on data structures: Traversing, Searching, Insertion, Deletion, Sorting, and Merging, with suitable examples for each.

1. Traversing

Access

2. Searching

Finding

3. Insertion

Adding

4. Deletion

Remov

5. Sorting

Arrangi

6. Merging

Combin

Q6. Discuss time complexity considerations for basic operations on data structures.

Time complexity measures how algorithm execution time grows as input size n increases:

| Data Structure | Access | Search | Insertion | Deletion |

* **Trade-off Analysis:** Arrays offer instant $O(1)$ random access by index but suffer slow $O(n)$ insertions. Linked Lists sacrifice random access ($O(n)$) to achieve fast $O(1)$ dynamic insertion and deletion.

Topic 4: Arrays & Multi-Dimensional Arrays

Q7. Explain in detail how a 2-D array is represented in memory using row-major and column-major order. Derive the address calculation formula for both.

A 2-D array $A[M][N]$ with lower bounds LBR, LBC, Base Address B, and element size w:

1. Row-Major Order (RMO)

Element

Equation:

Address

2. Column-Major Order (CMO)

Element

Equation:

Address

Q8. What is a sparse matrix? Explain its representation using triplet/array representation with an example.

1. Definition

A Sparse matrix has many zero elements.

2. Triplet Representation

Instead of storing the entire matrix, we store only the non-zero elements in a triplet table.

Example Matrix (4 x 4):

Equation:

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix}$$

Triplet Table:

| Row | Column | Value |

Q9. Explain multi-dimensional arrays with memory representation and address calculation with a numerical example.

1. Numerical Problem

Given a

Find the address of A[15][40].

2. Given Data

* **LB

3. Calculation

Equation:

Address

Equation:

Address

Equation:

Address

Equation:

Address

Answer: Memory Address of A[15][40] = 2460.

Topic 5: String Manipulation

Q10. Explain various string manipulation operations (concatenation, comparison, substring, reverse) with algorithms/examples.

1. Concatenation: Appends string `S2` to end of `S1`.

* **Algo

Q11. Write and explain an algorithm for pattern matching (Naive/Brute-force method) with an example.

1. Naive Pattern Matching Algorithm

Algorithm

2. Numerical Example

* **Te

Q12. Discuss string storage representations (fixed length vs linked list representation) in detail.

| Feature | Fixed-Length Array Representation | Variable Linked List Representation |

| :--- | :-

Topic 6: Self-Referential Structures with Pointers

Q13. Explain self-referential structures in detail with syntax, memory representation, and a diagram showing how nodes are linked.

A Self-Referential Structure is a C structure that contains at least one pointer member pointing to a structure of the same type:

```
struct Node {
```

int data

Memory Layout Diagram:

Node 1

Q14. Write a program/algorithm to create and traverse a simple linked list using self-referential structures.

```
#include <stdio.h>
```

```
#include
```

```
struct Node {
```

```
int data;
```

```
void traverse(struct Node *head) {
```

```
struct Node *temp = head;
```

```
int main() {
```

```
// Dynamic Memory Allocation
```

```
head->data = 100;
```

```
head->next = NULL;
```

```
second->data = 200;
```

```
second->next = head;
```

```
third->data = 300;
```

```
third->next = second;
```

```
traverse(head);
```

```
return 0;
```

```
---
```

Q15. Discuss the advantages of self-referential structures over arrays for dynamic data handling.

1. Dynamic Memory Allocation: Memory is allocated at runtime as needed via `malloc()`, eliminating static array capacity limits and memory wastage.

2. Efficient Traversal: Self-referential structures allow for easy traversal of the data, as each node contains a pointer to the next node, simplifying the traversal process.